GigScore

An alternative-data credit decision engine for people with little or no credit history — what it is, how it works, how well it works, and where its limits are.

In one paragraph. GigScore helps a lender decide on small two-wheeler loans for self-employed and gig workers, half of whom have no credit-bureau score. It turns what a loan officer knows into a 300–900 score, an approve / refer / decline decision and three plain-language reasons. After the loan is given, it reads the borrower's bank or UPI statement (with consent) and flags trouble before an EMI is missed. It was built and tested on 233,154 real loans from an Indian NBFC (L&T Financial Services, 2018).

Generated 29 October 2026 by `docs/make_explainer.py`. Every number in this document is read from the project's own outputs (feature table, saved models, configuration and the reports written by the code in `src/`).

Data: L&T Vehicle Loan Default Prediction (Kaggle mamtadhaker/lt-vehicle-loan-default-prediction), released for the LTFS Data Science FinHack (Analytics Vidhya, 2019). Statement samples used in the app are SYNTHETIC and labelled as such. The Pune rider survey workbook in the project folder is simulated and is not used by any model.

Contents

1. What GigScore is 3

2. The problem it solves 3

3. The data 4

4. How it works, end to end 5

5. Features: what the model looks at 5

6. The models 7

7. How well it works 8

8. The score and the decision bands 10

9. Fairness 10

10. Explaining every decision 11

11. Watch: the early-warning layer 13

12. The application 13

13. Running it and deploying it 15

14. Limits, and what it takes to use it for real 15

15. Problems found and fixed during the build 16

16. Glossary 17

1. What GigScore is

GigScore is a credit decision engine: software a lender (a bank, an NBFC or a fintech) plugs into its loan process. It is not a lender itself. It has three parts that share one code base:

Part	What it does	Who uses it
Score (at application)	Takes an application and returns a 300–900 score, the probability the borrower misses the first EMI, a band (approve / refer / decline) and three reasons.	Credit officer, or the lender's loan system through the API
Watch (after disbursal)	Reads 8–12 weeks of the borrower's bank / UPI statement and returns a green / amber / red flag, the signals that triggered it and a suggested next step.	Collections and customer-care teams
Portfolio, About, Admin	Totals across all decisions; the model card, validation report and fairness results; runtime settings (cut-offs, model, kill switch, rules).	Risk head, auditors, regulators, the team running it

Who it is for. Delivery riders, cab drivers and other self-employed people who want to finance a two-wheeler but are declined or overcharged because they have no CIBIL score. In the L&T data, 50.2% of borrowers had no bureau score at all and 54.7% were self-employed.

What makes it more than a model. It is built the way a lender would need it: a test on loans from a later month than the training data, calibrated probabilities, reasons a customer can be told, a fairness check, an audit log of every decision, a fallback model (kill switch), monitoring for drift, and settings that change without touching code.

2. The problem it solves

In India, lenders lean heavily on the credit-bureau score (CIBIL and others). A person who has never borrowed formally has no score, so the lender either declines them or treats them as high risk. Gig workers are hit hardest: their income is real but irregular and paid into UPI wallets and bank accounts, which a bureau never sees.

The data shows the bureau's blind spot directly. For the 49,748 October 2018 applicants with no bureau score, a model built only on bureau data ranks risk with an AUC of **0.507** — no better than tossing a coin (0.5). It cannot tell the safe ones from the risky ones, so a lender relying on it has to treat them all the same.

GigScore uses information the lender already collects at the dealership — loan-to-value, the vehicle, employment type, existing loans if any — and ranks the same no-score applicants with an AUC of **0.611**. That ranking is what lets a lender approve the safer ones instead of declining everybody.

3. The data

Source. 233,154 two-wheeler loans disbursed by L&T Financial Services between August and October 2018, with 41 columns: loan amount, vehicle price, loan-to-value, branch, dealer, vehicle maker, area, state, date of birth, employment type, which ID documents were shared, the bureau score and its description, and the applicant's existing loans.

What is predicted. `loan_default` = 1 means the borrower **missed the first EMI** on its due date. Overall 21.7% did. This is early-payment default — the earliest sign of repayment stress — which is also exactly what the Watch layer looks for later in the loan.

Disbursal month	Loans	First-EMI default rate	Role
2018-08	68,002	21.6%	training
2018-09	66,788	19.2%	training
2018-10	98,364	23.5%	test (out-of-time)

Who is in it.

- No bureau score at all (score = 0): 50.2%, defaulting at 23.1% vs 20.3% for scored borrowers.
- No *usable* score (score 0 or a 'Not Scored' placeholder): 55.7%.
- Self-employed: 54.7%, defaulting at 22.8%.

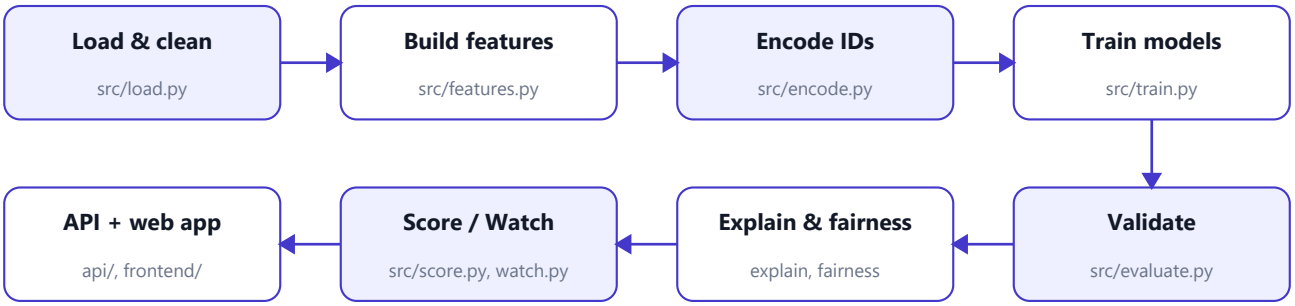
Quirks the code handles (file `src/load.py`):

- Dates use two-digit years. A birth year of '84' is 1984 but pandas reads '45' as 2045, so birth years after 2025 are moved back a century. Disbursal dates are recent and are never shifted.
- A bureau score of 0 means *no score*, not a bad score. It is kept as a flag and the score is treated as missing.
- Rows described as 'Not Scored: …' carry placeholder codes 11–18 instead of real scores. These are also treated as missing; the reason for not being scored is kept as its own category, because it predicts risk.
- Account ages are text such as '1yrs 11mon' and are converted to months.
- Some loan amounts are negative or absurd (up to 1,000,000,000). Amount columns are clipped at the 1st and 99th percentile of their non-zero values, using the training months only.

Honesty rule. Cleaning, feature screening, tuning, calibration and the decision cut-offs use August–September only; October 2018 is the test month. One choice did look at October: the decision to ship the fair model was made after comparing fairness options on October loans (Section 9). That was a policy choice about equal treatment, not a search for a higher score, and the fair model's October accuracy is lower than the alternative's.

4. How it works, end to end

The pipeline is a chain of small Python modules. Each reads the previous step's output from disk and writes its own, so any step can be re-run on its own.



Step	What happens	Output
Load & clean	Parse dates, fix the century rule, convert '1yrs 11mon', flag no-score, clip amounts.	a clean frame
Build features	Turn raw columns into 48 risk features in five families (Section 5).	features.parquet
Encode IDs	Replace branch / dealer / maker / area IDs by their past default rate, out-of-fold.	encoders.pkl
Train	Scorecard + LightGBM on four feature sets, tuned with 5-fold cross-validation inside Aug–Sep.	models/*.pkl
Validate	Measure on October: accuracy with confidence ranges, calibration, drift, business value, cut-offs.	validation_report.md, thresholds.yaml
Explain & fairness	Per-applicant reasons (SHAP); approval and error gaps by age band and state.	reports, model card
Score / Watch	One application → score, band, reasons. One statement → flag, triggers, action.	live decisions
API + web app	FastAPI serves decisions and logs every one; the Astro site is the user interface.	the running product

5. Features: what the model looks at

Features are grouped into five families borrowed from how credit analysts think (capacity, asset quality, management, earnings, liquidity) plus context. The table shows the strongest by **information value** (IV: how well a single feature separates defaulters from non-defaulters; above 0.1 is medium, below 0.02 is too weak for a scorecard).

Feature	Meaning	IV	In shipped model?
supplier_te	Dealer's past default rate (out-of-fold)	0.105	no — stands in for state
pincode_te	Area's past default rate	0.078	no — stands in for state
bureau_score	Bureau score (missing if none)	0.076	yes
branch_te	Branch's past default rate	0.072	no — stands in for state
ltv	Loan as % of the vehicle price	0.072	yes
down_payment_share	Share of the price paid upfront	0.068	no — duplicate
score_desc_grp	Bureau risk grade or reason for no score	0.062	yes

Feature	Meaning	IV	In shipped model?
new_emi_est	Estimated new EMI (1.5%/month, 36 months)	0.051	yes
total_emi	Existing EMIs + new EMI	0.042	yes
pri_sanctioned_amount	Total of past loans sanctioned	0.030	yes
pri_disbursed_amount	Total of past loans disbursed	0.029	no — duplicate
branch_n	Branch loan volume	0.029	no — stands in for state

Leakage-safe target encoding. Branch, dealer, maker and area have hundreds to thousands of codes. Each code is replaced by the default rate of its past loans, shrunk towards the overall rate when the code has few loans (smoothing m = 50). For the training rows this is done *out-of-fold*: the data is split into five parts and each row is encoded only from the other four, so no row ever sees its own outcome. New applications use the mapping from all training rows.

Deliberately left out. Age, age band and state are protected attributes: they are stored for the fairness check and never fed to the model. The employee who logged the loan is an operational artefact and is dropped. Disbursal month is not built, because the training months (Aug, Sep) never include October.

Ideas tested and rejected. Before the final training, six more features, averaging three models, and blending LightGBM with the scorecard were tried with cross-validation on Aug–Sep only. None improved AUC by the required 0.002, so none was adopted:

Experiment	CV AUC	Gain
baseline (current combined set)	0.6765	+0.0000
+ supplier_mfr_te	0.6767	+0.0001
+ branch_mfr_te	0.6767	+0.0002
+ branch_supplier_te	0.6769	+0.0004
+ ltv_vs_dealer	0.6772	+0.0006
+ dealer_noscore_share	0.6757	-0.0008
+ asset_vs_dealer	0.6767	+0.0002
all candidates together	0.6772	+0.0007
baseline, 3-seed bagging	0.6769	+0.0004
blend LightGBM 1.00 + scorecard 0.00	0.6765	+0.0000

6. The models

Two kinds of model are trained, each on four feature sets, giving eight models. Only two are used in production; the rest exist to measure what each kind of data contributes.

Model A — the scorecard (the fallback)

The classic, regulator-friendly design. Each feature is cut into up to ten bins; each bin is replaced by its **weight of evidence** (WoE = log of the share of good borrowers in the bin divided by the share of bad ones). A logistic regression then weighs the features, and the result is converted to points. A credit officer can read the whole model as a points table. Any feature whose weight comes out with the wrong sign (which would make its points run against the data) is removed and the model refitted.

The shipped scorecard `scorecard-fair-2026.10` uses 7 features: bureau_score, score_desc_grp, hist_len_m, pri_sanctioned_amount, ltv, new_emi_est, total_emi.

Model B — LightGBM (the main model)

A gradient-boosted tree model: hundreds of small decision trees, each correcting the previous ones. It captures interactions a scorecard cannot. Three safeguards keep it defensible:

- **Monotone constraints**: a higher loan-to-value, more delinquencies, more overdue loans or more enquiries can never lower the predicted risk, and more ID documents can never raise it.
- **Tuning by cross-validation**: Optuna tried 40 settings, each judged by 5-fold cross-validation inside Aug–Sep. The shipped model has 428 trees with num_leaves = 42, min_child_samples = 93, feature_fraction = 0.439, lambda_l2 = 26.619.
- **Calibration**: raw scores are mapped to real probabilities with isotonic regression fitted on out-of-fold predictions, so '12%' means about 12 in 100 such borrowers miss the first EMI.

The four feature sets

Set	Features	What it answers
bureau	22	How far can credit-bureau data alone go?
alternative	20	How far can non-bureau data alone go?
combined	43	Everything — the most accurate model (the alternative)
fair	32	Everything except features that stand in for state — **shipped**

The **fair** set removes area and branch encodings, dealer encodings and the ID-document flags. Section 9 explains why.

7. How well it works

All results are on the **98,364 October 2018 loans**, which the models never saw. Ranges are 95% confidence intervals from 500 bootstrap resamples. AUC is the probability that a randomly chosen defaulter gets a higher risk score than a randomly chosen non-defaulter: 0.5 is guessing, 1.0 is perfect.

Model	All applicants	No bureau score	Self-employed
Shipped (fair) LightGBM	0.637 (0.633–0.641)	0.611 (0.606–0.617)	0.631 (0.626–0.636)
All-features LightGBM	0.658 (0.654–0.662)	0.643 (0.638–0.649)	0.655 (0.649–0.660)
Fallback scorecard	0.597 (0.593–0.601)	0.580 (0.574–0.585)	0.590 (0.585–0.596)
Bureau data only	0.588 (0.584–0.592)	0.507 (0.505–0.510)	0.583 (0.578–0.588)

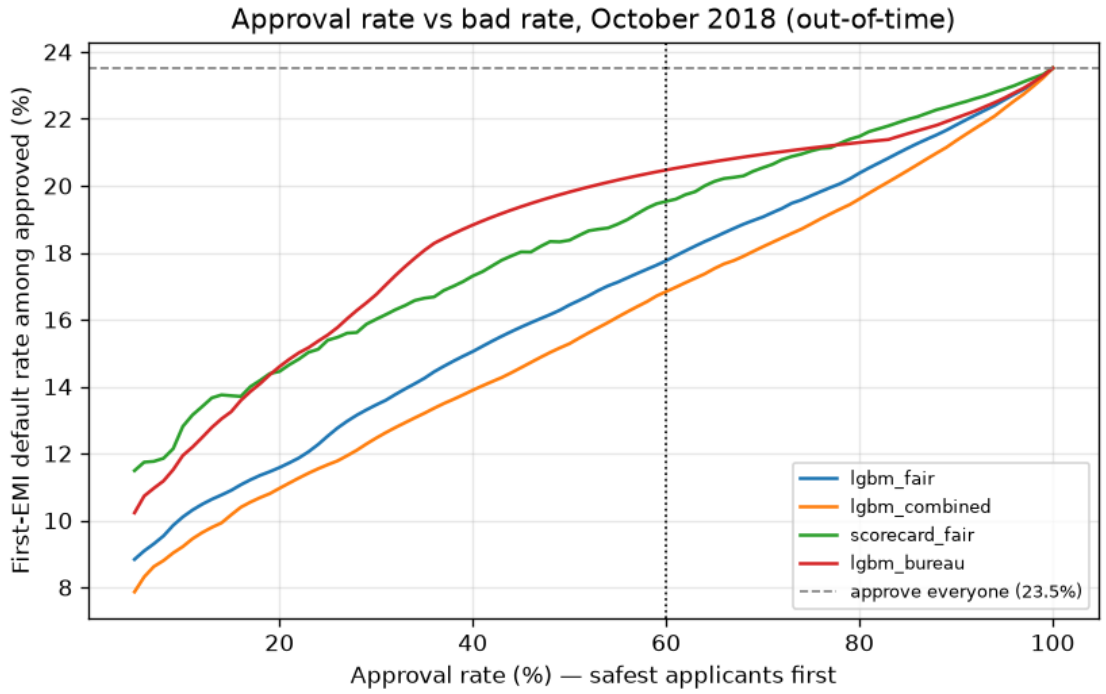
- **The headline:** for applicants with no bureau score, the shipped model's AUC is 0.611 against 0.507 for bureau data alone.
- **Is that good?** On this dataset, yes. First-EMI default is a noisy outcome and the data holds no income information; the project's build guide puts the best public competition results at around 0.67–0.68 (for a model that also uses age and location). An AUC above 0.75 here would signal a data leak, and none was found.
- **Random vs time split:** refitting the same settings on a random split of all three months gives 0.646 for the shipped model — higher, because October leaks into training. The out-of-time number is the honest one.

What a lender gets from it

AUC is abstract; a lender cares about how many people it can approve and how many of them repay. The table shows, at the default rate the bureau-only model reaches, how many more applicants the shipped model can approve:

Bureau-only approves	Its default rate	GigScore approves	Extra loans (of 98,364)
50%	19.8%	74.0%	+23,567
60%	20.5%	80.0%	+19,703
70%	20.9%	82.7%	+12,463

For no-score applicants specifically: keeping defaults at or below the training-period rate of 20.4%, the shipped model can approve 68% of them; the bureau-only model 0.1%.



Approval rate vs default rate among approved applicants, October 2018. Lower is better. Source: src/evaluate.py.

Calibration and monthly recalibration

October was riskier than the training months (23.5% vs 20.4%). No model trained on August–September can know that in advance, so the predicted probabilities run low in October while the *ranking* stays intact. The fix a lender would use is a monthly 'calibration offset': once the first-EMI outcomes of last month's loans are known, shift every probability by a constant in log-odds.

Back-tested within October (offset fitted on loans from 1–15 Oct, applied to 16–31 Oct): mean predicted probability 20.5% → 21.0% against 24.3% observed; worst decile gap 4.6 → 4.0 points. It helps, but risk kept rising through October, so a half-month-old offset closes only part of the gap. The offset is a setting (`calibration_offset`) that an admin can change in the app.

Stability

The **population stability index** (PSI) compares the score distribution between periods: below 0.10 is stable, above 0.25 needs investigation. The shipped model's score PSI is 0.002 for October and 0.004 for November 2018 (the unlabelled test.csv), measured against August–September.

8. The score and the decision bands

The probability of default (PD) is converted to points: **600 points at 4:1 good-to-bad odds** (a 20% PD), and every 50 points doubles the odds. Higher is safer; the result is clipped to 300–900.

$$\text{Score} = 600 + 50 \times \log_2\left(\frac{(1 - PD)}{PD} / 4\right)$$

PD (chance of missing the first EMI)	Score
5%	712
10%	658
15%	625
20%	600
30%	561
40%	529

Cut-offs. Approve if the score is at least **585**, refer to a credit officer if at least **569**, otherwise decline. They were set so that the safest 60% of August–September applicants would be approved and the next 20% referred, then applied unchanged to October:

Band	Share of October applicants	Actual first-EMI default
Approve	57.9%	17.5%
Refer	22.0%	27.9%
Decline	20.1%	36.1%

These cut-offs are a proposal. A lender chooses its own point on the approval curve; the values live in `config/thresholds.yaml` and can be changed from the Admin page without touching code.

9. Fairness

The model never sees age or state, but other features can stand in for them (a 'proxy'). The check looks at outcomes: approval rate and error rates by age band and by state, with gaps above 5 percentage points flagged. Two error rates matter: the share of defaulters who still get approved, and the share of good borrowers who get turned away (the second is the one a regulator worries about most).

Age band	Loans	Default rate	Approval rate	Gap vs all
18-24	18,706	25.6%	55.5%	-2.4 pp
25-34	37,292	24.2%	58.8%	+0.9 pp
35-44	25,465	22.1%	57.5%	-0.4 pp
45-54	13,634	22.0%	58.5%	+0.7 pp
55+	3,267	20.1%	61.3%	+3.4 pp

Age: 0 of 5 bands flagged. **State:** 3 of 18 states flagged for the shipped model.

Why the fair model is shipped

The all-features model leaned on area, branch and dealer, and on which ID documents were shared — all of which differ a lot between states. Good borrowers in some states were turned away far more often. Each row below removes more of these features and approves the safest 60%:

Model	AUC	Default rate, approved	Largest state gap	States flagged
lgbm-combined-2026.10 (all features)	0.658	16.8%	34.0 pp	9 of 18
without area & branch	0.652	17.1%	28.9 pp	10 of 18
without area, branch & dealer	0.639	17.7%	20.1 pp	7 of 18
without area, branch, dealer & ID documents	0.636	17.8%	8.6 pp	3 of 18
lgbm-fair-2026.10 (shipped; own tuning)	0.637	17.8%	8.5 pp	3 of 18

The trade: some accuracy and slightly more defaults among approved loans, for much smaller differences between states. For a product whose purpose is fair access for people the bureau cannot see, that is the right default. A lender can switch to the all-features model from the Admin page; that choice belongs with its risk and compliance teams.

One open point. With location features gone, the fair model leans more on employment type: 'Applicant is self-employed' appears among the reasons for many referred and declined applicants (Section 10). Self-employed borrowers did default more often in this data, so the signal is real, but for a product built for gig workers it deserves a deliberate decision by the lender — keep it, cap its effect, or remove it and accept the accuracy cost.

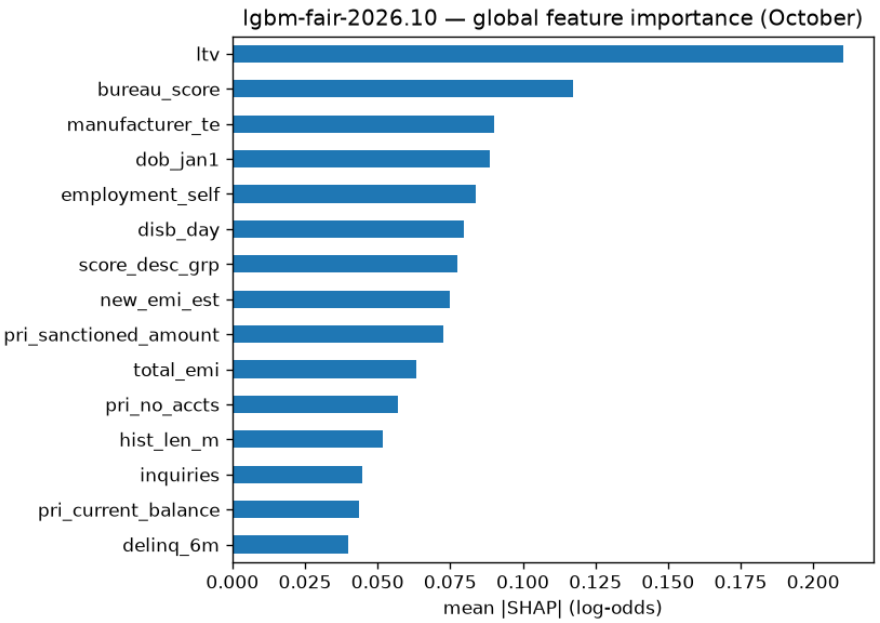
10. Explaining every decision

Each decision comes with three plain-language reasons. For LightGBM they come from **SHAP** values, which split a prediction into how much each feature pushed the risk up or down for this particular applicant (LightGBM computes them exactly with its built-in TreeSHAP). For the scorecard they come from the points table. A declined or referred applicant sees the three strongest factors that *raised* risk; an approved one the three strongest that *lowered* it.

Every feature has a sentence written for customers, e.g. loan-to-value → 'The loan is a high share of the vehicle price'. Process features (disbursal timing, a placeholder birth date) stay in the model but are never shown as reasons, because they say nothing the customer can understand or act on.

Most frequent reasons on October 2018 applicants (shipped model):

Band	Most frequent reasons (share of applicants in the band)
Approve	A good share of the vehicle price was paid upfront (55%) The credit-bureau score is good (34%) This vehicle make has a better repayment history (30%) Applicant is not self-employed (26%) Credit-bureau risk grade is low (A–D) (25%) Total monthly EMIs are manageable (23%)
Refer	The loan is a high share of the vehicle price (53%) The new EMI is large (53%) The credit-bureau score is low (45%) Applicant is self-employed (34%) This vehicle make has a weaker repayment history (25%) Total monthly EMIs, including this loan, are high (24%)
Decline	The loan is a high share of the vehicle price (69%) The credit-bureau score is low (49%) The new EMI is large (46%) This vehicle make has a weaker repayment history (28%) Applicant is self-employed (22%) Total monthly EMIs, including this loan, are high (18%)



Global importance of each feature in the shipped model (mean absolute SHAP value). Source: src/explain.py.

11. Watch: the early-warning layer

The L&T data has no cash-flow history, so the early-warning layer starts as transparent rules — the way most collections teams already work — and is written so a trained model can replace the rules later without changing the app. It reads 8–12 weeks of statement rows (date, credit, debit, balance, optionally a category such as rent or utility and a status such as late or bounced).

Consent first. The API refuses to read a statement unless the request says the borrower consented. Only the statement is used — no contacts, call logs or device data, in line with RBI's Digital Lending Directions.

Signal	Amber if	Red if
Income trend (last 4 weeks vs 12-week average)	below 80%	below 60%
Weeks with no income (last 8)	1	2 or more
Lowest balance in the 5 days before the EMI date	below 1.0× the EMI	below 0.5× the EMI
Rent / utility paid late or bounced (last 90 days)	1	2 or more
All EMIs as a share of the last 28 days' income	above 35%	above 50%
Share of each inflow spent within 48 hours	above 70%	above 90%

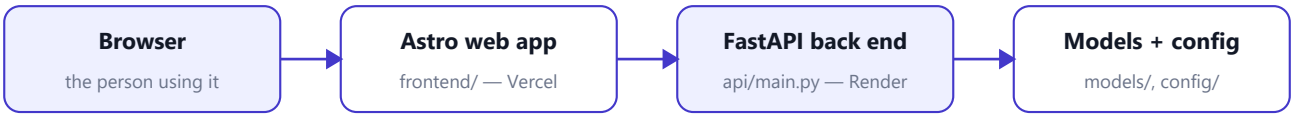
Combining signals. Any red signal, or 3 or more amber ones → **red**. 2 amber → **amber**. Otherwise green. Fewer than 8 weeks of statement → 'insufficient data'.

Suggested actions. Green: none. Amber: a reminder call — or, if the only problem is a low balance just before the due date while income holds up, an offer to move the EMI date. Red: contact the borrower about restructuring before an EMI is missed.

> **Status of the rules.** The thresholds come from the build guide and the survey design. They have not yet been validated on real rider statements. The three demo statements in the app (steady, slipping, stressed) are SYNTHETIC and only show that the rules behave sensibly. Treat flags as prompts for a human call, not as decisions.

12. The application

The product has two parts that can be hosted separately:



- **Front end (Astro).** Six pages built once into static files; every number on them is fetched live from the API in the browser, so the site always shows the current model, settings and log.
- **Back end (FastAPI).** Loads the models once, scores applications, runs the watch rules, writes every decision to a SQLite audit log, and serves the reports and charts.
- **Configuration** (`config/thresholds.yaml`, `config/watch_rules.yaml`) holds every decision threshold, so behaviour changes without code changes.

The screens

Page	What you do there	What you see
Overview	Start here.	Headline results: test size, no-score ranking power, how many more no-score applicants can be approved, extra loans at equal risk.
Score an application	Enter loan amount, vehicle price, age, employment, documents, bureau score (or why there is none) and, optionally, existing loans.	A gauge with the 300–900 score and band zones, the chance of missing the first EMI, how the applicant ranks, three reasons, and where they sit on the approval curve.
Connect & Watch	Tick consent; load a SYNTHETIC sample or upload a statement CSV; enter the EMI and due day.	Green / amber / red, the suggested action, each signal with what was observed, and weekly income.
Portfolio	Review.	Applications by band, approval rate, expected default, watched loans by flag, the October 2018 reference book, and every logged decision with its exact inputs and output.
About the model	Read.	Accuracy with confidence ranges, the fairness comparison, and the full model card, validation report, feature report and data profile.
Admin	Enter the admin token.	Change the model, cut-offs, calibration offset, kill switch and watch rules live; reload retrained models; see the change log.

API endpoints

Endpoint	Purpose
POST /score	Score one application → score, pd, band, reasons, model_version, percentile, explain_id
POST /watch	Check one loan's statement (requires consent: true) → flag, triggers, rules, suggested_action
GET /samples/{profile}	A SYNTHETIC 12-week statement: steady, slipping or stressed
GET /decisions, /decisions/{id}	The audit log; retrieve any decision to answer a customer complaint
GET /portfolio, /model, /about/{doc}	Totals, model reference data and settings, and the reports
PUT /admin/thresholds	Change model, cut-offs, calibration offset or kill switch (admin token)
PUT /admin/watch-rules	Change watch thresholds (admin token)
POST /admin/reload	Load retrained model files without restarting (admin token)
GET /health	Which model is live and whether the fallback is active

Safeguards built in

- **Audit log:** every score, watch check and settings change is stored with inputs, output, model version and time, and test / SYNTHETIC decisions are marked.
- **Kill switch:** if the main model fails, or an admin turns on 'force fallback' (for example after a drift alarm), the scorecard takes over and the response says so.
- **Protected attributes** are stored for fairness reporting only; the test suite checks that changing the state never changes the score.

- **Admin access** needs a token set on the server; without it, admin endpoints are disabled. Every change needs a written reason.
- **Input checks:** impossible inputs (age outside 18–75, loan above 120% of the vehicle price, bureau score outside 300–900) are rejected rather than scored.

13. Running it and deploying it

Run on your own computer

```
.venv\Scripts\uvicorn.exe api.main:app --reload
→ web app at http://localhost:8000/ , API docs at http://localhost:8000/docs
```

Rebuild everything from the raw data

```
python -m src.profiling → python -m src.features → python -m src.train (≈45 min) → python -m
src.evaluate → pytest -q
```

Put it on the internet

- Put the project in a GitHub repository (git needs to be installed first).
- **Render (API):** New → Blueprint → the repository. render.yaml sets up the service. Set ADMIN_TOKEN (a long random string you keep private) and ALLOWED_ORIGINS (the Vercel address).
- **Vercel (web app):** New Project → the repository → root directory 'frontend'. Set PUBLIC_API_URL to the Render address.
- Both services need accounts in your name; deployment is done by you.

14. Limits, and what it takes to use it for real

- **The data is from 2018 and one lender.** Default levels and loan sizes are dated. Before real use the pipeline must be retrained on the deploying lender's own loans — a configuration change, not a rewrite.
- **The borrowers are 'gig-like', not confirmed gig workers:** self-employed two-wheeler buyers. The planned Pune rider survey and consented rider statements are what would connect the model to actual riders. The survey file in the project folder is simulated and was not used.
- **Accuracy is modest** (AUC about 0.64–0.66) because first-EMI default is noisy and the data has no income. The next real gain needs cash-flow data (bank statements through the Account Aggregator framework, platform payouts).
- **Probabilities drift** when the market shifts, as October 2018 showed. Monthly recalibration is built in but only partly closes a fast-moving gap.
- **The Watch rules are unvalidated** until they are tested on real statements.
- **Only a regulated lender can lend.** The realistic path is a shadow-mode pilot with an NBFC: score live applications in parallel for 8–12 weeks without affecting decisions, then compare outcomes by band.

15. Problems found and fixed during the build

Every one of these was caught by a test or by checking a result against the raw data, and fixed before any number in this document was produced:

Problem	Effect before the fix
Each row's dealer / branch / area average included its own outcome	A small leak that flattered accuracy
Binning collapsed mostly-zero columns into one bin	Delinquency and enquiry features looked useless
Stability check compared two different encodings	False drift alarm (3.6 instead of 0.003)
Correlation screen dropped the no-score flag	Lost the subgroup flag the project depends on
Clipping by plain percentiles	Co-applicant columns flattened to zero or capped at ₹50,000
A scorecard weight with the wrong sign	Points ran against the data for credit-history length
Approval-curve maths split tied scores	Bureau-only looked able to approve 7.9% of no-score applicants (0.1%)
Validation rows used for tuning, early stopping and calibration	Optimistic validation scores
Cut-offs chosen on the test month	Optimistic band default rates
Century rule applied to disbursal dates	A loan disbursed in 2026 would be read as 1926

16. Glossary

Where to find more: reports/validation_report.md (every test and number), reports/model_card.md (the model card), reports/progress.md (what was built, step by step), README.md (how to run and deploy).

Term	Meaning
AUC	Chance a random defaulter is scored riskier than a random non-defaulter. 0.5 = guessing, 1 = perfect.
Gini / KS	Other ranking measures: Gini = 2 × AUC − 1; KS = largest gap between the score distributions of defaulters and non-defaulters.
PD	Probability of default — here, the chance of missing the first EMI.
Out-of-time test	Testing on a later month than the training data, as real deployment works.
Out-of-fold	A prediction or encoding for a row made by a model that never saw that row.
Calibration	Making '12%' mean that about 12 in 100 such borrowers default.
WoE / IV	Weight of evidence (how a bin's good/bad mix compares with the whole) and information value (how strongly a feature separates good from bad).
PSI	Population stability index — how much a distribution has shifted between two periods.
SHAP	A method that splits one prediction into each feature's contribution; used for reasons.
Monotone constraint	A rule forcing the model's risk to only rise (or only fall) as a feature grows.
Kill switch	Switching to the simpler fallback model when the main model fails or drifts.
NBFC	Non-banking financial company — the kind of lender that originates most new-to-credit loans in India.
PAR 30 / 90	Portfolio at risk: share of loans more than 30 / 90 days overdue (needs repayment history the L&T data does not contain).